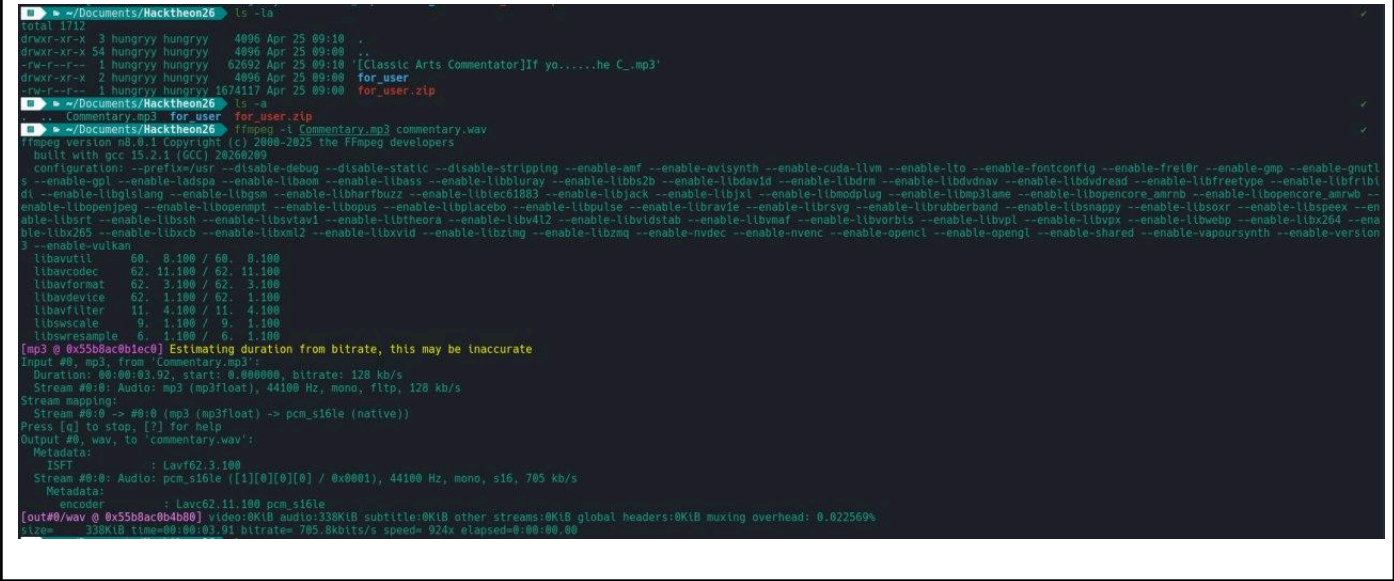


2026 HACKTHEON SEJONG

팀 이름 Team Name	🇸🇬🇮🇩🇵🇹_NUT Flaggers
문제 이름 Question	Voice Over

ish.audio/app/m/bb3536eac3594cfa90585a95c5c6dd16/



2026 HACKTHEON SEJONG

Text to Speech

Classic Arts Commentator

If you listen to a UNIX shell, can you hear the C?

+ Add Speaker

Tags ▾

7.9k credits remaining

63 / 500 bytes

✦ Enhance

Generate speech

Sample A



0:00/0:03

Sample B



0:00/0:04

```

~/Documents/Hacktheon26 % ls -la
total 1712
drwxr-xr-x 3 hungry hungry 4096 Apr 25 09:10 .
drwxr-xr-x 54 hungry hungry 4096 Apr 25 09:00 ..
-rw-r--r-- 1 hungry hungry 62692 Apr 25 09:10 [Classic Arts Commentator]If you.....he C_.mp3'
drwxr-xr-x 2 hungry hungry 4096 Apr 25 09:00 for_user
-rw-r--r-- 1 hungry hungry 1674117 Apr 25 09:00 for_user.zip
~/Documents/Hacktheon26 % ls -la
total 16K
-rw-r--r-- 1 hungry hungry 16K Apr 25 09:00 for_user.zip
~/Documents/Hacktheon26 % ffmpeg -i Commentary.mp3 commentary.wav
ffmpeg version 48.1 Copyright (c) 2000-2025 The FFmpeg developers
built with gcc 15.2.1 (GCC) 2020200
configuration: --prefix=/usr --disable-debug --disable-static --disable-stripping --enable-amf --enable-avsynth --enable-cuda-llvm --enable-fontconfig --enable-freib --enable-gmp --enable-gnutls
--enable-gpl --enable-ladspa --enable-lbass --enable-lbliburai --enable-lbbs2b --enable-lbdav1d --enable-lbdrm --enable-lbdrvnav --enable-lbdrvread --enable-lbfsretype --enable-lbfrbri
d1 --enable-lbglslang --enable-lbgsm --enable-lbharfbuzz --enable-lbtecu1883 --enable-lbjack --enable-lbjxl --enable-lbmoadplug --enable-lbmpp3lame --enable-lbopencore_amrnb --enable-lbopencore_amrwb
--enable-lbopenjpeg --enable-lbopenmpt --enable-lbopus --enable-lblacpaco --enable-lbpulse --enable-lbrawfi --enable-lbraws --enable-lbbruberband --enable-lbtsnappy --enable-lbspx --en
able-lbts2 --enable-lbts2l --enable-lbtheora --enable-lb412 --enable-lbvidstabil --enable-lbvmaf --enable-lbvorbis --enable-lbvpx --enable-lbwop --enable-lb264 --en
able-lb265 --enable-lb266 --enable-lb267 --enable-lb268 --enable-lb269 --enable-lb270 --enable-lb271 --enable-lb272 --enable-lb273 --enable-lb274 --enable-lb275 --enable-lb276 --enable-lb277 --enable-lb278
--enable-lb279 --enable-lb280 --enable-lb281 --enable-lb282 --enable-lb283 --enable-lb284 --enable-lb285 --enable-lb286 --enable-lb287 --enable-lb288 --enable-lb289 --enable-lb290 --enable-lb291 --enable-lb292
--enable-lb293 --enable-lb294 --enable-lb295 --enable-lb296 --enable-lb297 --enable-lb298 --enable-lb299 --enable-lb300 --enable-lb301 --enable-lb302 --enable-lb303 --enable-lb304 --enable-lb305 --enable-lb306
--enable-lb307 --enable-lb308 --enable-lb309 --enable-lb310 --enable-lb311 --enable-lb312 --enable-lb313 --enable-lb314 --enable-lb315 --enable-lb316 --enable-lb317 --enable-lb318 --enable-lb319 --enable-lb320
--enable-lb321 --enable-lb322 --enable-lb323 --enable-lb324 --enable-lb325 --enable-lb326 --enable-lb327 --enable-lb328 --enable-lb329 --enable-lb330 --enable-lb331 --enable-lb332 --enable-lb333 --enable-lb334
--enable-lb335 --enable-lb336 --enable-lb337 --enable-lb338 --enable-lb339 --enable-lb340 --enable-lb341 --enable-lb342 --enable-lb343 --enable-lb344 --enable-lb345 --enable-lb346 --enable-lb347 --enable-lb348
--enable-lb349 --enable-lb350 --enable-lb351 --enable-lb352 --enable-lb353 --enable-lb354 --enable-lb355 --enable-lb356 --enable-lb357 --enable-lb358 --enable-lb359 --enable-lb360 --enable-lb361 --enable-lb362
--enable-lb363 --enable-lb364 --enable-lb365 --enable-lb366 --enable-lb367 --enable-lb368 --enable-lb369 --enable-lb370 --enable-lb371 --enable-lb372 --enable-lb373 --enable-lb374 --enable-lb375 --enable-lb376
--enable-lb377 --enable-lb378 --enable-lb379 --enable-lb380 --enable-lb381 --enable-lb382 --enable-lb383 --enable-lb384 --enable-lb385 --enable-lb386 --enable-lb387 --enable-lb388 --enable-lb389 --enable-lb390
--enable-lb391 --enable-lb392 --enable-lb393 --enable-lb394 --enable-lb395 --enable-lb396 --enable-lb397 --enable-lb398 --enable-lb399 --enable-lb400 --enable-lb401 --enable-lb402 --enable-lb403 --enable-lb404
--enable-lb405 --enable-lb406 --enable-lb407 --enable-lb408 --enable-lb409 --enable-lb410 --enable-lb411 --enable-lb412 --enable-lb413 --enable-lb414 --enable-lb415 --enable-lb416 --enable-lb417 --enable-lb418
--enable-lb419 --enable-lb420 --enable-lb421 --enable-lb422 --enable-lb423 --enable-lb424 --enable-lb425 --enable-lb426 --enable-lb427 --enable-lb428 --enable-lb429 --enable-lb430 --enable-lb431 --enable-lb432
--enable-lb433 --enable-lb434 --enable-lb435 --enable-lb436 --enable-lb437 --enable-lb438 --enable-lb439 --enable-lb440 --enable-lb441 --enable-lb442 --enable-lb443 --enable-lb444 --enable-lb445 --enable-lb446
--enable-lb447 --enable-lb448 --enable-lb449 --enable-lb450 --enable-lb451 --enable-lb452 --enable-lb453 --enable-lb454 --enable-lb455 --enable-lb456 --enable-lb457 --enable-lb458 --enable-lb459 --enable-lb460
--enable-lb461 --enable-lb462 --enable-lb463 --enable-lb464 --enable-lb465 --enable-lb466 --enable-lb467 --enable-lb468 --enable-lb469 --enable-lb470 --enable-lb471 --enable-lb472 --enable-lb473 --enable-lb474
--enable-lb475 --enable-lb476 --enable-lb477 --enable-lb478 --enable-lb479 --enable-lb480 --enable-lb481 --enable-lb482 --enable-lb483 --enable-lb484 --enable-lb485 --enable-lb486 --enable-lb487 --enable-lb488
--enable-lb489 --enable-lb490 --enable-lb491 --enable-lb492 --enable-lb493 --enable-lb494 --enable-lb495 --enable-lb496 --enable-lb497 --enable-lb498 --enable-lb499 --enable-lb500 --enable-lb501 --enable-lb502
--enable-lb503 --enable-lb504 --enable-lb505 --enable-lb506 --enable-lb507 --enable-lb508 --enable-lb509 --enable-lb510 --enable-lb511 --enable-lb512 --enable-lb513 --enable-lb514 --enable-lb515 --enable-lb516
--enable-lb517 --enable-lb518 --enable-lb519 --enable-lb520 --enable-lb521 --enable-lb522 --enable-lb523 --enable-lb524 --enable-lb525 --enable-lb526 --enable-lb527 --enable-lb528 --enable-lb529 --enable-lb530
--enable-lb531 --enable-lb532 --enable-lb533 --enable-lb534 --enable-lb535 --enable-lb536 --enable-lb537 --enable-lb538 --enable-lb539 --enable-lb540 --enable-lb541 --enable-lb542 --enable-lb543 --enable-lb544
--enable-lb545 --enable-lb546 --enable-lb547 --enable-lb548 --enable-lb549 --enable-lb550 --enable-lb551 --enable-lb552 --enable-lb553 --enable-lb554 --enable-lb555 --enable-lb556 --enable-lb557 --enable-lb558
--enable-lb559 --enable-lb560 --enable-lb561 --enable-lb562 --enable-lb563 --enable-lb564 --enable-lb565 --enable-lb566 --enable-lb567 --enable-lb568 --enable-lb569 --enable-lb570 --enable-lb571 --enable-lb572
--enable-lb573 --enable-lb574 --enable-lb575 --enable-lb576 --enable-lb577 --enable-lb578 --enable-lb579 --enable-lb580 --enable-lb581 --enable-lb582 --enable-lb583 --enable-lb584 --enable-lb585 --enable-lb586
--enable-lb587 --enable-lb588 --enable-lb589 --enable-lb590 --enable-lb591 --enable-lb592 --enable-lb593 --enable-lb594 --enable-lb595 --enable-lb596 --enable-lb597 --enable-lb598 --enable-lb599 --enable-lb600
--enable-lb601 --enable-lb602 --enable-lb603 --enable-lb604 --enable-lb605 --enable-lb606 --enable-lb607 --enable-lb608 --enable-lb609 --enable-lb610 --enable-lb611 --enable-lb612 --enable-lb613 --enable-lb614
--enable-lb615 --enable-lb616 --enable-lb617 --enable-lb618 --enable-lb619 --enable-lb620 --enable-lb621 --enable-lb622 --enable-lb623 --enable-lb624 --enable-lb625 --enable-lb626 --enable-lb627 --enable-lb628
--enable-lb629 --enable-lb630 --enable-lb631 --enable-lb632 --enable-lb633 --enable-lb634 --enable-lb635 --enable-lb636 --enable-lb637 --enable-lb638 --enable-lb639 --enable-lb640 --enable-lb641 --enable-lb642
--enable-lb643 --enable-lb644 --enable-lb645 --enable-lb646 --enable-lb647 --enable-lb648 --enable-lb649 --enable-lb650 --enable-lb651 --enable-lb652 --enable-lb653 --enable-lb654 --enable-lb655 --enable-lb656
--enable-lb657 --enable-lb658 --enable-lb659 --enable-lb660 --enable-lb661 --enable-lb662 --enable-lb663 --enable-lb664 --enable-lb665 --enable-lb666 --enable-lb667 --enable-lb668 --enable-lb669 --enable-lb670
--enable-lb671 --enable-lb672 --enable-lb673 --enable-lb674 --enable-lb675 --enable-lb676 --enable-lb677 --enable-lb678 --enable-lb679 --enable-lb680 --enable-lb681 --enable-lb682 --enable-lb683 --enable-lb684
--enable-lb685 --enable-lb686 --enable-lb687 --enable-lb688 --enable-lb689 --enable-lb690 --enable-lb691 --enable-lb692 --enable-lb693 --enable-lb694 --enable-lb695 --enable-lb696 --enable-lb697 --enable-lb698
--enable-lb699 --enable-lb700 --enable-lb701 --enable-lb702 --enable-lb703 --enable-lb704 --enable-lb705 --enable-lb706 --enable-lb707 --enable-lb708 --enable-lb709 --enable-lb710 --enable-lb711 --enable-lb712
--enable-lb713 --enable-lb714 --enable-lb715 --enable-lb716 --enable-lb717 --enable-lb718 --enable-lb719 --enable-lb720 --enable-lb721 --enable-lb722 --enable-lb723 --enable-lb724 --enable-lb725 --enable-lb726
--enable-lb727 --enable-lb728 --enable-lb729 --enable-lb730 --enable-lb731 --enable-lb732 --enable-lb733 --enable-lb734 --enable-lb735 --enable-lb736 --enable-lb737 --enable-lb738 --enable-lb739 --enable-lb740
--enable-lb741 --enable-lb742 --enable-lb743 --enable-lb744 --enable-lb745 --enable-lb746 --enable-lb747 --enable-lb748 --enable-lb749 --enable-lb750 --enable-lb751 --enable-lb752 --enable-lb753 --enable-lb754
--enable-lb755 --enable-lb756 --enable-lb757 --enable
```

2026 HACKTHEON SEJONG

http://3.37.31.209:8000

// VOICE CLONE CHALLENGE

Clone the target voice and speak the given sentence

Target Sentence

Chuck Norris can dereference NULL.

Submit Your Audio

Click or drag & drop your WAV file here
commentary.wav

VERIFY

ACCESS GRANTED

Speaker Similarity 0.8872 / 0.8

Text Similarity 0.9592 / 0.8

Transcript "If you listen to a unique shell, can you hear the C?"

hacktheon2026{b7d30e21e4106a6ca4d451a218f15a97}

Using an online AI vice generator, we generate the voice and submit it accordingly. had to convert it to wav using ffmpeg.

2026 HACKTHEON SEJONG

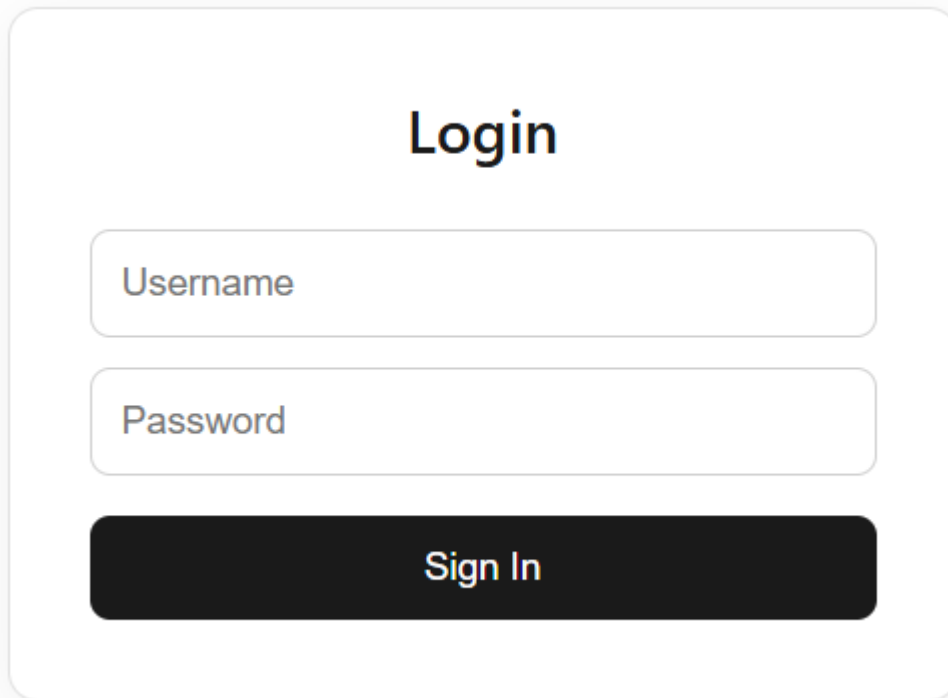
팀 이름 Team Name	 _NUT Flaggers
문제 이름 Question	Simple-sqli

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

We were given this website:

<http://43.203.99.42:8000/>

Looks like a normal webpage:



Looking at source code:

```
query = (  
    "SELECT username, role, secret FROM users "  
    f"WHERE username = '{username}' AND password = '{password}' "  
)
```

This means normal SQL injection on username parameter, in this case “**admin’ --**” will cause SQL injection to view admin account, ignore password parameter:



Welcome, admin!

User information retrieved from database

USERNAME	admin
ROLE	ADMIN
SECRET	hacktheon2026{d0nt_f0rget_the_s1ngle_qu0te}

← Back to login

Flag: hacktheon2026{d0nt_f0rget_the_s1ngle_qu0te}

2026 HACKTHEON SEJONG

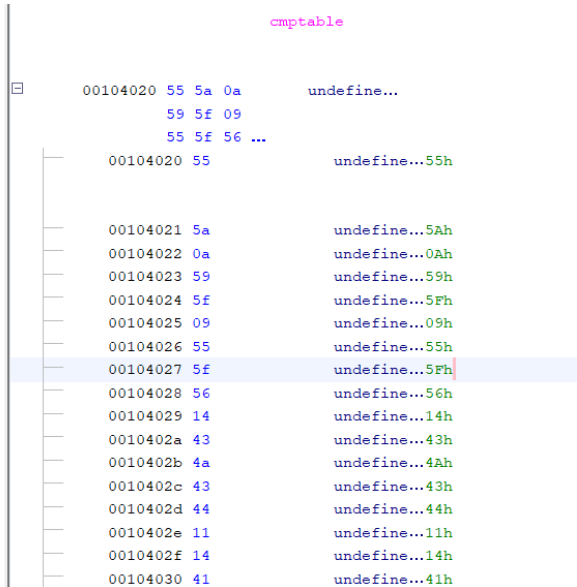
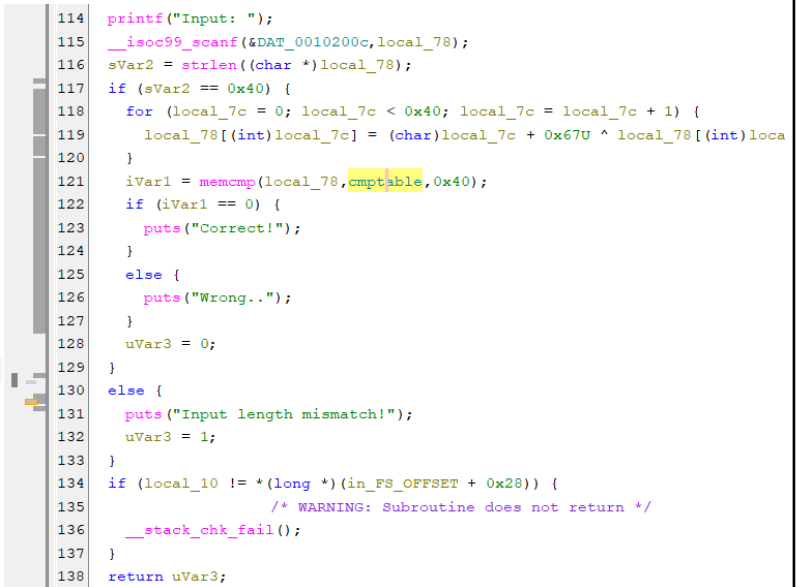
팀 이름 Team Name	 _NUT Flaggers
문제 이름 Question	Recover it!

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

We were given a ELF program “prob”. Upon opening with Ghidra and decompiling it, we see the main program details:

```
printf("Input: ");
__isoc99_scanf(&DAT_0010200c, local_78);
sVar2 = strlen((char *)local_78);
if (sVar2 == 0x40) {
    for (local_7c = 0; local_7c < 0x40; local_7c = local_7c + 1) {
        local_78[(int)local_7c] = (char)local_7c + 0x67U ^ local_78[(int)local_7c];
    }
    iVar1 = memcmp(local_78, cmptable, 0x40);
    if (iVar1 == 0) {
        puts("Correct!");
    }
    else {
        puts("Wrong..");
    }
    uVar3 = 0;
}
else {
    puts("Input length mismatch!");
    uVar3 = 1;
}
if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
}
return uVar3;
}
```

Program asked for some input, and then XORing with “0x67”. Since this is a simple reversing challenge, what we need is find the comparing array, then just XOR with “0x67” again:

 cmptable 00104020 55 5a 0a undefined... 59 5f 09 55 5f 56 ... 00104020 55 undefined...55h 00104021 5a undefined...5Ah 00104022 0a undefined...0Ah 00104023 59 undefined...59h 00104024 5f undefined...5Ph 00104025 09 undefined...09h 00104026 55 undefined...55h 00104027 5f undefined...5Ph 00104028 56 undefined...56h 00104029 14 undefined...14h 0010402a 43 undefined...43h 0010402b 4a undefined...4Ah 0010402c 43 undefined...43h 0010402d 44 undefined...44h 0010402e 11 undefined...11h 0010402f 14 undefined...14h 00104030 41 undefined...41h	 <pre>114 printf("Input: "); 115 __isoc99_scanf(&DAT_0010200c, local_78); 116 sVar2 = strlen((char *)local_78); 117 if (sVar2 == 0x40) { 118 for (local_7c = 0; local_7c < 0x40; local_7c = local_7c + 1) { 119 local_78[(int)local_7c] = (char)local_7c + 0x67U ^ local_78[(int)loca 120 } 121 iVar1 = memcmp(local_78, cmptable, 0x40); 122 if (iVar1 == 0) { 123 puts("Correct!"); 124 } 125 else { 126 puts("Wrong.."); 127 } 128 uVar3 = 0; 129 } 130 else { 131 puts("Input length mismatch!"); 132 uVar3 = 1; 133 } 134 if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) { 135 /* WARNING: Subroutine does not return */ 136 __stack_chk_fail(); 137 } 138 return uVar3;</pre>
--	--

Just by looking at the bytes in cmptable, we are able to feed the bytes in and XOR with 0x67 to get the flag:

2026 HACKTHEON SEJONG

main.py		Run	Output
<pre>1- cmptable = [2 0x55, 0x5a, 0x0a, 0x59, 0x5f, 0x09, 0x55, 0x5f, 3 0x56, 0x14, 0x43, 0x4a, 0x43, 0x44, 0x11, 0x14, 4 0x41, 0x48, 0x4c, 0x1e, 0x42, 0x1a, 0x19, 0x1c, 5 0x1c, 0xb9, 0xe3, 0xe3, 0xba, 0xe5, 0xe7, 0xb1, 6 0xb6, 0xec, 0xbf, 0xe8, 0xb2, 0xbd, 0xba, 0xbb, 7 0xeb, 0xa6, 0xf3, 0xa1, 0xf1, 0xa4, 0xa4, 0xa0, 8 0xf4, 0xac, 0xa0, 0xf9, 0xff, 0xa5, 0xa9, 0xa8, 9 0xad, 0x94, 0xc7, 0x97, 0x97, 0x91, 0xc6, 0x95 10] 11 12 result = [] 13 14 for i in range(64): 15 decoded_byte = cmptable[i] ^ (i + 0x67) 16 result.append(decoded_byte) 17 18 # print as string 19 print("Decoded string:") 20 print("".join(chr(b) for b in result)) 21 22 # also print hex (in case of weird chars)</pre>		Run	Decoded string: 22c34e819d2800db605d9fdb9ba9ab71d6b9175d6b3b016c49cd94624f545c3 Hex output: 32 32 63 33 34 65 38 31 39 64 32 38 30 30 64 62 36 30 35 64 39 66 64 62 63 39 62 61 35 61 62 37 31 64 36 62 39 31 37 35 64 36 62 33 62 30 31 36 63 34 39 63 64 39 34 36 32 34 66 35 34 35 63 33 === Code Execution Successful ===

Flag: hacktheon2026{22c34e819d2800db605d9fdb9ba9ab71d6b9175d6b3b016c49cd94624f545c3}

2026 HACKTHEON SEJONG

팀 이름 Team Name	SGIDV_NUT Flaggers
문제 이름 Question	Cathedral of the Last Candle
문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)	

Overview

This challenge is a 5x8 board puzzle over GF(3). The visible symbols map to values $'.' = 0$, $'*' = 1$, $'\sim' = 2$. Each cell has one bonded neighbor. Using 'ring' at the current cell transforms $[a, b]$ to $[a+b, a+2b] \bmod 3$, and 'hush' is the inverse transform $[2a+2b, 2a+b] \bmod 3$.

Key observation

If all bonds were between visible cells only, the global state could not be driven to all zero because both operators are invertible. The altar is special. At position (4,7), 'ring' increments the altar value by $+1 \bmod 3$ and 'hush' decrements it by $-1 \bmod 3$. That hidden altar behavior is what makes full extinction possible.

Discovery automation

The solver first reveals the full board with a snake walk. Then it revisits every cell in snake order and applies 'ring' twice. Comparing the local visible cross before and after those two operations is enough to infer the unique bonded direction for that cell. In live runs, the bond layout changes per session, but it is always local and forms a directed tree rooted at the altar.

Solve method

After recovering the tree, the solver performs bottom-up dynamic programming. Each subtree is summarized as a small transducer from parent value to parent value, while forcing the subtree root to zero. The DP state tracks solved children, current node value, and current parent value. This avoids greedy mistakes where a node must use its own edge before some child subtree becomes solvable.

Execution

The script cathedral_solve.py connects to the service, learns the tree, computes the minimum command sequence, simulates it locally for sanity, replays it against the service, walks to the altar, and sends 'pray'. A successful live run produced a completely dark board and returned the flag below.

Result

hacktheon2026{2c6ccbb801652250fbdbe2650de78ae74bd153fc84e4d69905b7b65c623a28a33aa09b020817ddb251f415bd8e59d343cd72f62dd1d60d6986b7b1ea5468538078b06a15b0118e2}

```
kuri@kuri:~/1-CTF/korea/pwn2/misc$ python3 cathedral_solve.py
discovered budget 131 visible nonzero 40
*****
*****
*****
*****
*****
hidden 2
plan 118 moves 78 ops 40
sim pos (4, 7) sim visible zero True
.....
.....
.....
.....
.....
hidden 2

All candles grow dark.
The cathedral draws a long, quiet breath.
hacktheon2026{cb8481a11b772b7934d15e287e7bab63d39d14ef2577a6082d524881184b0e8a57c307abf3b48032a468c031d20ef246d7f5e98ccb57221db875cd684c82948d8d26f5be48606505}

kuri@kuri:~/1-CTF/korea/pwn2/misc$ |
```

```
import re
import socket
from collections import deque
from heapq import heappop, heappush
```

```
HOST = "3.35.223.94"
```

```
PORT = 9000
```

```
R, C = 5, 8
```

```
HIDDEN = ("altar", "hidden")
```

```
ALTAR = (4, 7)
```

```
SYM_TO_VAL = {".": 0, "*": 1, "~": 2}
```

```
VAL_TO_SYM = ".*~"
```

```
POS_RE = re.compile(r"Pos:\((\d+),(\d+)\)")
BUD_RE = re.compile(r"Budget:(\d+)")
CELL_RE = re.compile(r"\[([.*~])\]|([.*~])")
```

```
class Client:
```

```
    def __init__(self):
```

```
        self.s = socket.create_connection((HOST, PORT), timeout=5)
```

```
        self.pos = None
```

```
        self.budget = None
```

```
        self.rows = None
```

```
        self.last = self._recv()
```

```
        self._parse(self.last)
```

```
    def close(self):
```

```
        self.s.close()
```

```
    def _recv(self):
```

```
        data = b""
```

```
        while not data.endswith(b"> "):
```

```
chunk = self.s.recv(4096)

    if not chunk:
        break

    data += chunk

return data.decode("utf-8", "replace")


def _parse(self, text):
    m = POS_RE.search(text)
    if m:
        self.pos = (int(m.group(1)), int(m.group(2)))
    m = BUD_RE.search(text)
    if m:
        self.budget = int(m.group(1))
    rows = []
    for line in text.splitlines():
        toks = []
        for m in CELL_RE.finditer(line):
            toks.append(m.group(1) or m.group(2))
        if len(toks) >= C:
            rows.append(toks[:C])
    if len(rows) == R:
        self.rows = rows


def cmd(self, command):
    self.s.sendall((command + "\n").encode())
    self.last = self._recv()
    self._parse(self.last)
    return self.last


def visible(self):
    out = {}
    for r in range(R):
        for c in range(C):
```

```
ch = self.rows[r][c]
    if ch != "?":
        out[(r, c)] = SYM_TO_VAL[ch]
    return out
```

```
def neighbors(p):
    r, c = p
    for dr, dc, cmd in [(-1, 0, "go n"), (1, 0, "go s"), (0, -1, "go w"), (0, 1, "go e")]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < R and 0 <= nc < C:
            yield (nr, nc), cmd
```

```
def shortest_path(a, b):
    q = deque([(a, [])])
    seen = {a}
    while q:
        p, path = q.popleft()
        if p == b:
            return path
        for np, cmd in neighbors(p):
            if np not in seen:
                seen.add(np)
                q.append((np, path + [cmd]))
    raise RuntimeError("no path")
```

```
def go_to(cl, target, state=None):
    for command in shortest_path(cl.pos, target):
        cl.cmd(command)
        if state is not None:
            state.update(cl.visible())
```

```
def explore(cl):
    state = {}
```

```
state.update(cl.visible())

snake = []
for r in range(R):
    cols = range(C) if r % 2 == 0 else range(C - 1, -1, -1)
    for c in cols:
        snake.append((r, c))
for p in snake:
    go_to(cl, p, state)
return state

def print_state(state):
    for r in range(R):
        print("".join(VAL_TO_SYM[state[(r, c)]] for c in range(C)))
    if HIDDEN in state:
        print("hidden", state[HIDDEN])

def diff_visible(before, after):
    return {p: (before[p], after[p]) for p in before.keys() & after.keys() if before[p] != after[p]}

def adj(p):
    r, c = p
    out = []
    for nr, nc in ((r - 1, c), (r + 1, c), (r, c - 1), (r, c + 1)):
        if 0 <= nr < R and 0 <= nc < C:
            out.append((nr, nc))
    return out

def probe_bonds():
    cl = Client()
    state = explore(cl)
    print("initial")
    print_state(state)
    print("pos", cl.pos, "budget", cl.budget)
```

```

bonds = {}
for r in range(R):
    for c in range(C):
        p = (r, c)
        go_to(cl, p, state)
        before = dict(state)
        before_vis = cl.visible()
        cl.cmd("ring")
        state.update(cl.visible())
        d = diff_visible(before_vis, cl.visible())
        changed_neighbors = [q for q in d if q != p and abs(q[0] - r) + abs(q[1] - c) == 1]
        if len(changed_neighbors) == 1:
            bonds[p] = changed_neighbors[0]
        else:
            # A second ring disambiguates the a+b == 0 case for nonzero candles.
            before_vis = cl.visible()
            cl.cmd("ring")
            state.update(cl.visible())
            d2 = diff_visible(before_vis, cl.visible())
            changed_neighbors = [q for q in d2 if q != p and abs(q[0] - r) + abs(q[1] - c) ==
1]

            if len(changed_neighbors) == 1:
                bonds[p] = changed_neighbors[0]
            else:
                bonds[p] = None
                print("ambiguous", p, "d1", d, "d2", d2)
            print(p, "->", bonds[p], "budget", cl.budget)
print("after probes")
print_state(state)
print("bonds")
for r in range(R):
    row = []
    for c in range(C):

```

```
q = bonds[(r, c)]

    if q is None:
        row.append("?")
    elif q[0] < r:
        row.append("^")
    elif q[0] > r:
        row.append("v")
    elif q[1] < c:
        row.append("<")
    else:
        row.append(">")
    print(" ".join(row))
cl.close()

def apply_ring_state(state, p, q):
    a, b = state[p], state[q]
    state[p] = (a + b) % 3
    state[q] = (a + 2 * b) % 3

def infer_hidden_after_two_rings(a0, a1, a2):
    b0 = (a1 - a0) % 3
    b1 = (a0 + 2 * b0) % 3
    if (a1 + b1) % 3 != a2:
        raise RuntimeError(f"hidden inference mismatch: {a0=} {a1=} {a2=} {b0=} {b1=}")
    b2 = (a1 + 2 * b1) % 3
    return b0, b2

def infer_bond_local(p, before_vis, after1_vis, after2_vis):
    if p == ALTAR:
        a0 = before_vis[p]
        if after1_vis[p] == (a0 + 1) % 3 and after2_vis[p] == (a0 + 2) % 3:
            return HIDDEN, 1
```

```
raise RuntimeError(f"cannot infer altar behavior from {before_vis=} {after1_vis=}
{after2_vis=}")
```

```
a0 = before_vis[p]
candidates = adj(p)
```

```
ok = []
```

```
for q in candidates:
```

```
    if q == HIDDEN:
```

```
        a1 = after1_vis[p]
```

```
        a2 = after2_vis[p]
```

```
        try:
```

```
            b0, _ = infer_hidden_after_two_rings(a0, a1, a2)
```

```
        except RuntimeError:
```

```
            continue
```

```
        s = dict(before_vis)
```

```
        s[HIDDEN] = b0
```

```
        apply_ring_state(s, p, HIDDEN)
```

```
        exp1 = {k: s[k] for k in before_vis}
```

```
        apply_ring_state(s, p, HIDDEN)
```

```
        exp2 = {k: s[k] for k in before_vis}
```

```
        if exp1 == after1_vis and exp2 == after2_vis:
```

```
            ok.append((q, b0))
```

```
        continue
```

```
b0 = before_vis[q]
```

```
s = dict(before_vis)
```

```
apply_ring_state(s, p, q)
```

```
exp1 = {k: s[k] for k in before_vis}
```

```
apply_ring_state(s, p, q)
```

```
exp2 = {k: s[k] for k in before_vis}
```

```
if exp1 == after1_vis and exp2 == after2_vis:
```

```
    ok.append((q, None))
```

```
if len(ok) != 1:
    raise RuntimeError(f"cannot infer bond for {p}: {ok} from {before_vis=} {after1_vis=}
{after2_vis=}")
return ok[0]

def discover(cl):
    state = explore(cl)
    bonds = {}
    snake = []
    for r in range(R):
        cols = range(C) if r % 2 == 0 else range(C - 1, -1, -1)
        for c in cols:
            snake.append((r, c))
    for p in snake:
        go_to(cl, p, state)
        before_vis = cl.visible()
        cl.cmd("ring")
        after1_vis = cl.visible()
        cl.cmd("ring")
        after2_vis = cl.visible()
        q, hidden0 = infer_bond_local(p, before_vis, after1_vis, after2_vis)
        bonds[p] = q
        state = dict(state)
        if q == HIDDEN:
            state[HIDDEN] = hidden0
        apply_ring_state(state, p, q)
        apply_ring_state(state, p, q)
        state.update(after2_vis)
    return state, bonds

def nonzero_count(state):
    return sum(1 for p, v in state.items() if p != HIDDEN and v)
```

```
def visible_zero(state):  
    return all(state[(r, c)] == 0 for r in range(R) for c in range(C))
```

```
def build_children(bonds):  
    children = {p: [] for p in bonds}  
    for p, q in bonds.items():  
        if q in children:  
            children[q].append(p)  
    return children
```

```
def double(x):  
    return (2 * x) % 3
```

```
def solve_tree_dp(state, bonds):  
    children = build_children(bonds)  
    post = []
```

```
    def dfs(u):  
        for v in children[u]:  
            dfs(v)  
        post.append(u)
```

```
    dfs(ALTAR)
```

```
    dp = {}
```

```
    choice = {}
```

```
    for u in post:
```

```
        kid_list = children[u]
```

```
        k = len(kid_list)
```

```
        full = (1 << k) - 1
```

```
        if u == ALTAR:
```

```
            start = (0, state[u])
```

```
            dist = {start: 0}
```

```
            prev = {}
```

```

pq = [(0, start)]
while pq:
    cost, st = heappop(pq)
    if cost != dist[st]:
        continue
    mask, a = st
    # local altar ops
    for cmd, na in (("ring", (a + 1) % 3), ("hush", (a - 1) % 3)):
        ns = (mask, na)
        nc = cost + 1
        if nc < dist.get(ns, 10**9):
            dist[ns] = nc
            prev[ns] = (st, ("self", cmd))
            heappush(pq, (nc, ns))
    # solve one child subtree as an atomic action
    for i, ch in enumerate(kid_list):
        if mask & (1 << i):
            continue
        for a_out in range(3):
            sub = dp[ch].get((a, a_out))
            if sub is None:
                continue
            ns = (mask | (1 << i), a_out)
            nc = cost + sub
            if nc < dist.get(ns, 10**9):
                dist[ns] = nc
                prev[ns] = (st, ("child", ch, a, a_out))
                heappush(pq, (nc, ns))
goal = (full, 0)
dp[u] = {("root", "root"): dist.get(goal)}
choice[u] = {("root", "root"): (goal, prev)}
continue

```

```
table = {}
ctab = {}
for pin in range(3):
    start = (0, state[u], pin)
    dist = {start: 0}
    prev = {}
    pq = [(0, start)]
    while pq:
        cost, st = heappop(pq)
        if cost != dist[st]:
            continue
        mask, a, b = st
        for cmd, ns in (
            ("ring", (mask, (a + b) % 3, (a + 2 * b) % 3)),
            ("hush", (mask, (2 * a + 2 * b) % 3, (2 * a + b) % 3)),
        ):
            nc = cost + 1
            if nc < dist.get(ns, 10**9):
                dist[ns] = nc
                prev[ns] = (st, ("self", cmd))
                heappush(pq, (nc, ns))
    for i, ch in enumerate(kid_list):
        if mask & (1 << i):
            continue
        for a_out in range(3):
            sub = dp[ch].get((a, a_out))
            if sub is None:
                continue
            ns = (mask | (1 << i), a_out, b)
            nc = cost + sub
            if nc < dist.get(ns, 10**9):
                dist[ns] = nc
                prev[ns] = (st, ("child", ch, a, a_out))
```

```
heappush(pq, (nc, ns))

    for pout in range(3):
        goal = (full, 0, pout)
        if goal in dist:
            table[(pin, pout)] = dist[goal]
            ctab[(pin, pout)] = (goal, prev)

    dp[u] = table
    choice[u] = ctab
    return dp, choice, children

def step_to(a, b):
    if b[0] == a[0] - 1:
        return "go n"
    if b[0] == a[0] + 1:
        return "go s"
    if b[1] == a[1] - 1:
        return "go w"
    return "go e"

def emit_solution(choice, children, node, pin, pout=None):
    cmds = []
    key = ("root", "root") if node == ALTAR else (pin, pout)
    goal, prev = choice[node][key]
    seq = []
    st = goal
    while st in prev:
        pst, action = prev[st]
        seq.append(action)
        st = pst
    seq.reverse()
    for action in seq:
        if action[0] == "self":
            cmds.append(action[1])
```

```
continue
```

```
_, ch, child_pin, child_pout = action
cmds.append(step_to(node, ch))
cmds.extend(emit_solution(choice, children, ch, child_pin, child_pout))
cmds.append(step_to(ch, node))
```

```
return cmds
```

```
def simulate(state, bonds, cmds, pos=ALTAR):
```

```
    s = dict(state)
```

```
    p = pos
```

```
    for cmd in cmds:
```

```
        if cmd.startswith("go "):
```

```
            d = cmd.split()[1]
```

```
            r, c = p
```

```
            if d == "n":
```

```
                p = (r - 1, c)
```

```
            elif d == "s":
```

```
                p = (r + 1, c)
```

```
            elif d == "w":
```

```
                p = (r, c - 1)
```

```
            else:
```

```
                p = (r, c + 1)
```

```
            continue
```

```
    if p == ALTAR:
```

```
        if cmd == "ring":
```

```
            s[p] = (s[p] + 1) % 3
```

```
        elif cmd == "hush":
```

```
            s[p] = (s[p] - 1) % 3
```

```
        else:
```

```
            raise RuntimeError(cmd)
```

```
        continue
```

```
    q = bonds[p]
```

```
    if cmd == "ring":
```

```
apply_ring_state(s, p, q)
    elif cmd == "hush":
        apply_ring_state(s, p, q)
        apply_ring_state(s, p, q)
        apply_ring_state(s, p, q)
    else:
        raise RuntimeError(cmd)
return s, p

def reduce_and_pray():
    cl = Client()
    state, bonds = discover(cl)
    altar = ALTAR
    print("discovered budget", cl.budget, "visible nonzero", nonzero_count(state),
flush=True)
    print_state(state)
    dp, choice, children = solve_tree_dp(state, bonds)
    cmds = emit_solution(choice, children, altar, None, None)
    sim_state, sim_pos = simulate(state, bonds, cmds, cl.pos)
    print("plan", len(cmds), "moves", sum(1 for x in cmds if x.startswith('go ')), "ops",
sum(1 for x in cmds if not x.startswith('go ')), flush=True)
    print("sim pos", sim_pos, "sim visible zero", visible_zero(sim_state), flush=True)
    print_state(sim_state)
    for cmd in cmds:
        cl.cmd(cmd)
    out = cl.cmd("pray")
    print(out, flush=True)
    cl.close()

if __name__ == "__main__":
    reduce_and_pray()
```

